

# 开源软件基础

## Python 基础

Zhilei Ren



OSCAR Team, School of Software, Dalian University of Technology

November 15, 2017



# Outline

## 1 Python 环境安装

## 2 Hello world

## 3 变量、类型、基本流程控制

## 4 函数

## 5 递归

# 为什么选择 Python

# Python 版本

# Python 发行版

- vanilla Python
- Anaconda
- Python (x,y)
- Active Python

# Jupyter Notebook

# 本学期将涉及的 Python 知识

## 知识点

- 流程控制（结构化）
- 面向对象（皮毛）
- 数据获取
- 数据存储
- 数据可视化
- Web 开发（皮毛）

# Outline

- 1 Python 环境安装
- 2 Hello world
- 3 变量、类型、基本流程控制
- 4 函数
- 5 递归



# 第一个 Python 程序

```
$ipython3
ython 3.5.2+ (default, Sep 22 2016, 12:18:14)
Type ``copyright``, ``credits`` or ``license`` for more information.

IPython 5.1.0 -- An enhanced Interactive Python.
?                -> Introduction and overview of IPython's features.
%quickref        -> Quick reference.
help             -> Python's own help system.
object?         -> Details about 'object', use 'object??' for extra details.

In [1]:
```

# 第一个 Python 程序

```
In [1]: print("Hello world")  
Hello world
```

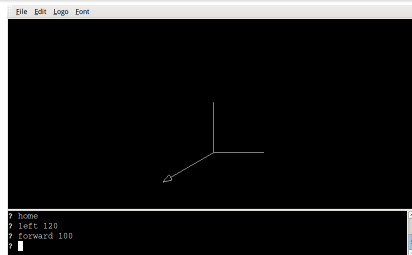
```
In [2]: print 1 + 2  
3
```

```
In [3]:
```

# Python Turtle

## Logo

Python Turtle 受 Logo 语言启发而来。



# Python Turtle

```
to tree :len :n
  if :n < 1 [stop]
  forward :len
  left 45
  tree (:len / 2) (:n - 1)
  right 90
  tree (:len / 2) (:n - 1)
  left 45
  back :len
end
```

# Outline

- 1 Python 环境安装
- 2 Hello world
- 3 变量、类型、基本流程控制**
- 4 函数
- 5 递归

# Outline

- 1 Python 环境安装
- 2 Hello world
- 3 变量、类型、基本流程控制
- 4 函数**
- 5 递归

# Outline

- 1 Python 环境安装
- 2 Hello world
- 3 变量、类型、基本流程控制
- 4 函数
- 5 递归

# 递归

To understand recursion, first you have to understand recursion.



# 阶乘

```
def factorial(n):  
    if n <= 1:  
        return 1  
    else:  
        return n * factorial(n - 1)
```

```
import turtle

def tree(length,n):
    """ paints a branch of a tree """
    if length < (length/n):
        return
    turtle.forward(length)
    turtle.left(45)
    tree(length * 0.5,length/n)
    turtle.right(90)
    tree(length * 0.5,length/n)
    turtle.left(45)
    turtle.backward(length)
    return
```

# 边界条件  
# 主干  
# 左转分叉  
# 左子树  
# 右转分叉  
# 右子树  
# 转回  
# 回到起点  
# 返回